Practical Skills

BMED365: Topic List F01–F14

BMED365

Spring 2026

Overview

1. **Jupyter Notebooks and Google Colab** (Jupyter, Colab)
2. **Python Fundamentals** (NumPy, Pandas, Matplotlib)
3. **Machine Learning with scikit-learn** (scikit-learn)
4. **Network Analysis with NetworkX** (NetworkX)
5. **Deep Learning with PyTorch** (PyTorch)
6. **AI Tools and LaTeX** (ChatGPT, Gemini, Claude, Overleaf)

F01: Run Jupyter Notebooks in Google Colab

What is Jupyter Notebooks?

- Interactive documents that combine code, text, and visualizations
- Run Python code cell by cell
- Standard in data science work

Google Colab:

- Free cloud-based Jupyter environment from Google
- No installation - runs in the browser
- Free GPU access (important for deep learning!)
- Integrated with Google Drive

Getting started:

1. Go to `colab.research.google.com`
2. Log in with Google account
3. "New notebook" or open from GitHub

Tip

Enable GPU: Runtime → Change runtime type → GPU

F02: Use Python variables, lists, and simple functions

Variables:

```
age = 45              # int
name = "Patient␣A"    # str
risk = 0.73           # float
is_sick = True        # bool
```

Lists:

```
symptoms = ["headache", "nausea", "fatigue"]
values = [1.2, 3.4, 5.6, 7.8]
symptoms.append("fever")  # Add element
```

Functions:

```
def calculate_bmi(weight, height):
    return weight / (height ** 2)

bmi = calculate_bmi(70, 1.75)  # -> 22.9
```

F03: Import and use libraries (numpy, pandas, matplotlib)

Importing libraries:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

NumPy:

- Numerical computations
- Arrays and matrices
- Mathematical functions

```
arr = np.array([1, 2, 3])
mean = np.mean(arr)
```

Pandas:

- Data manipulation
- DataFrames (tables)
- CSV, Excel I/O

```
df = pd.read_csv("data.csv")
df.head()
```

F04: Read and inspect datasets with pandas

Reading data:

```
df = pd.read_csv("patients.csv")
```

Inspecting data:

```
df.head()          # First 5 rows
df.info()          # Column types, null values
df.describe()      # Statistics for numeric columns
df.shape           # (number of rows, number of columns)
df.columns         # Column names
```

Filtering and selection:

```
# Select columns
df[["age", "diagnosis"]]

# Filter rows
elderly = df[df["age"] > 65]
```

F05: Train a simple model with scikit-learn

Typical ML workflow:

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# 1. Prepare data
X = df[["feature1", "feature2"]]
y = df["label"]

# 2. Split into train and test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# 3. Create and train model
model = DecisionTreeClassifier()
model.fit(X_train, y_train)

# 4. Evaluate
y_pred = model.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, y_pred):.2f}")
```

F06: Visualize results with matplotlib

Basic plotting:

```
import matplotlib.pyplot as plt

# Line chart
plt.plot([1, 2, 3, 4], [10, 20, 25, 30])
plt.xlabel("Time"); plt.ylabel("Value")
plt.title("My figure")
plt.show()
```

Histogram:

```
plt.hist(df["age"], bins=20)
plt.xlabel("Age")
plt.show()
```

Scatter plot:

```
plt.scatter(df["x"], df["y"])
plt.xlabel("X"); plt.ylabel("Y")
plt.show()
```

Seaborn

`import seaborn as sns` - Beautiful visualizations with simple code!

F07: Use NetworkX for simple network analysis

Create and manipulate graphs:

```
import networkx as nx

# Create graph
G = nx.Graph()
G.add_nodes_from(["P1", "P2", "P3", "P4"])
G.add_edges_from([("P1", "P2"), ("P2", "P3"), ("P1", "P3")])

# Basic analysis
print(f"Number of nodes: {G.number_of_nodes()}")
print(f"Number of edges: {G.number_of_edges()}")

# Centrality
deg_cent = nx.degree_centrality(G)
print(f"Degree centrality: {deg_cent}")

# Visualization
nx.draw(G, with_labels=True, node_color="lightblue")
plt.show()
```

F08: Build and train a model with PyTorch

Simple MLP in PyTorch:

```
import torch
import torch.nn as nn

class SimpleMLP(nn.Module):
    def __init__(self, input_size, hidden_size, num_classes):
        super().__init__()
        self.fc1 = nn.Linear(input_size, hidden_size)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        out = self.fc1(x)
        out = self.relu(out)
        out = self.fc2(out)
        return out

model = SimpleMLP(784, 128, 10)  # MNIST example
```

Lab 2

Full training with loss, optimizer, and training loop is covered in Lab 2.

F09: Use AI tools as coding assistants

Useful applications:

- **Explain code:** "What does this function do?"
- **Debugging:** "Why am I getting this error?"
- **Suggestions:** "Make this more efficient?"
- **Generate code:** "Write a function..."

Tips for effective use:

1. Be **specific** in your questions
2. Include **context**
3. Always **verify** AI code
4. Use as a **learning tool**

AI-powered IDEs (Integrated Development Environments)

IDE = Editor + compilation + debugger + tools in one. **AI-powered IDE** = IDE with LLM agents that can plan, write, test and validate code autonomously.
Significance for medical/biomedical research: Lowers the barrier to develop analysis tools, accelerates prototyping of ML models, and enables faster translation from research to clinic.

Cursor, Windsurf	Editors with agents, plan mode
Google Antigravity	Agent-first, multi-model (Gemini 3)
Replit Agent	Cloud-based IDE with AI agent
Lovable	Natural language → full-stack app
GitHub Copilot, JetBrains AI	AI assistants in existing IDEs

F10: Write documents with LaTeX/Overleaf

BibTeX (.bib file):

```
@article{smith2024,
  author  = {Smith, J.},
  title   = {AI in Medicine},
  journal = {Nature Med.},
  year    = {2024},
  volume  = {30},
  pages   = {123--130},
  doi     = {10.1038/s41591-
             024-01234-5}
}

@book{bishop2006,
  author    = {Bishop, C.},
  title     = {Pattern Recog.},
  publisher = {Springer},
  year      = {2006},
  isbn      = {978-0387310732}
}
```

In text: \cite{smith2024}

IMRAD structure:

```
\documentclass{article}
\usepackage{graphicx,booktabs}
\begin{document}
\title{Title}
\author{Author}
\maketitle
\begin{abstract} ... \end{abstract}
% Introduction
\section{Introduction}
Background and aim \cite{smith2024}.
% Materials and methods
\section{Methods}
Data collection and analysis.
% Results
\section{Results}
\begin{table}[h]
\begin{tabular}{lcc} \toprule
Model & AUC & F1 \\ \midrule
CNN & 0.92 & 0.88 \\ \bottomrule
\end{tabular}
\caption{Results}\label{tab:1}
\end{table}
% Discussion
\section{Discussion}
Implications and limitations.
\bibliography{refs}
\end{document}
```

Journals with LaTeX:

- PLOS Medicine
- Nature / Nat. Med.
- Bioinformatics
- The Lancet
- BMJ
- Elsevier (many)
- Springer Nature
- MDPI (open access)
- IEEE JBHI
- JMLR
- Frontiers

Overleaf Gallery

F11: Compose effective prompts for medical tasks

From Prompt Engineering to Context Engineering:

- **Prompt engineering:** Crafting good questions and instructions
- **Context engineering:** Designing the *entire context* the model uses

Components of context engineering:

1. **System prompt:** Role and basic instructions
2. **User prompt:** The specific question
3. **Retrieved context (RAG):** Medical records, guidelines, articles
4. **Tools:** Lab lookup, drug database, ICD-10
5. **Conversation history:** Previous dialog in the consultation
6. **Structured data:** JSON with lab values, vital signs

Medical significance

Rich context = more relevant, personalized, and safe AI responses

F12: Apply context engineering in practice

Example: Simple prompt vs. rich context

Simple prompt

```
What is the treatment for heart failure?
```

→ Generic answer, not personalized

Rich context

```
System:  Clinical decision support
Patient:  72 years, eGFR 45, Warfarin
RAG: ESC Guidelines 2023
Question:  Treatment changes?
```

→ Personalized, evidence-based answer

Skill levels:

Beginner:	Prompt formulation (role, format, examples)
Intermediate:	Structured system prompts with sections
Advanced:	RAG integration and tool use
Expert:	Multi-agent orchestration

F13: Choose the right LLM model for the task

Model strengths for medical tasks:

Task type	Recommended model	Rationale
Structured extraction	GPT-5.2	Good instruction following
Ethical dilemmas	Claude Opus 4.5	Nuanced reasoning
Image analysis	Gemini 3	Multimodal from the ground up
Long record analysis	Claude Opus 4.5	200K token context
Updated research	Gemini 3	Integrated real-time search
Tool integration	GPT-5.2	Good API for functions
Patient communication	Claude Opus 4.5	Empathetic, patient-centered
Calculations	Gemini 3	Strong in science

Practical recommendation

For critical tasks: Test on multiple models and compare output

F14: Compare output from different LLMs

Evaluation criteria for medical LLM output:

Quality dimensions:

1. **Medical accuracy**
   Is the information correct?
2. **Uncertainty handling**
   Is uncertainty expressed appropriately?
3. **Structured output**
   Are format instructions followed?
4. **Safety behavior**
   Is referral to physician recommended when needed?
5. **Clinical utility**
   Is the answer practically applicable?

Practical approach:

1. Define test case with ground truth
2. Send same prompt to 2–3 models
3. Evaluate against criteria (1–5 scale)
4. Document strengths/weaknesses
5. Choose model for production

Lab 3 Notebook 04

Contains ready-made experiments for model comparison

Summary: Practical Skills

Jupyter and Python:

- F01–F04: Colab, variables, lists, functions, pandas

Machine Learning:

- F05–F06: scikit-learn workflow, matplotlib visualization

Specialized:

- F07: NetworkX for network analysis
- F08: PyTorch for deep learning

Tools and LLM:

- F09–F10: AI assistants, LaTeX/Overleaf
- F11–F14: Context engineering, model selection, prompt optimization

QuickStart and Labs

All these skills are practiced through the QuickStart and Labs 0–3. Practice makes perfect!